A Deep Q-Network (DQN) Framework for Joint Optimization of EV Charging Station Placement and Vehicle Routing

Iakovos Ioannou*, Christophoros Christophorou†, Christina (Tanya) Politi§, Spyros Denazis‡, Marios Raspopoulos†, Vasos Vassiliou*¶

* Department of Computer Science, University of Cyprus and CYENS - Centre of Excellence, Cyprus
† University of Central Lancashire (UCLAN), Larnaca, Cyprus
‡ University of Patras, Greece
§ University of Peloponnese, Greece

***Abstract*—Rapid EV growth requires efficient charging infrastructure and routing for urban sustainability, vital for reducing congestion, wait times, and energy use while promoting renewables. This paper presents a two-stage Deep Q-Network (DQN) framework for optimal EV station placement and routing. A DQN placement model minimizes future route energy by selecting station locations; a second DQN guides vehicles by approximating cost-to-go functions. Our DQN approach outperforms baselines (Q-Learning, NN, DGNN, Random Walk) in energy consumption, travel time, and route success. DQN placement achieved 1.2387 kWh average energy and 6.34 km average distance. DQN routing delivered 1.2587 kWh average energy, 1.98 average hops, and 96.67% success, demonstrating our DRL's effectiveness for scalable, reliable, and energy-efficient EV infrastructure planning.**

***Index Terms*—Electric Vehicles (EVs), Deep Q-Network (DQN), Charging Station Placement, Route Optimization, Reinforcement Learning, Smart Mobility, Energy Efficiency**

I. Introduction

The accelerating global adoption of electric vehicles (EVs) is driving a transformative shift in sustainable urban transportation [1]. However, this transition introduces formidable challenges in optimizing limited charging infrastructure, especially in complex and dynamic urban environments [2]. High-density cityscapes exhibit erratic EV traffic patterns and spatially diverse charging demands [3], often resulting in long queues, station congestion, and critical energy-depletion events. These issues erode user trust and inhibit the scalability of EV ecosystems [4]. This work targets the dual optimization challenge central to EV-based smart mobility: (i) the strategic placement of a fixed number of charging stations ($K = 3$ in our experimental setup) within an urban road network, represented as a directed graph $G = (V, E)$, where nodes represent candidate charging locations and edges encode travel and energy costs; and (ii) the real-time routing of EVs to these stations to minimize energy consumption, travel time, and distance [5]. The complexity of real-world traffic networks, characterized by dynamic topologies, varying vehicle battery state-of-charge, and unpredictable demands, necessitates intelligent and scalable learning-based solutions [6].

This study introduces a comprehensive and modular two-phase solution that harnesses Deep Reinforcement Learning (DRL) and supervised machine learning for efficient EV charging system design. Phase one addresses charging station placement through various strategies, including a Deep Q-Network (DQN) [7] that learns placement actions via reward feedback based on simulated routing energy [8]. It is supplemented by other machine learning approaches for placement, such as Neural Networks (NN) [9] and Graph Neural Networks (GNNs) [10], which directly score or determine placements based on routing outcomes [11]. In phase two, a separately trained DQN routing model approximates energy-optimal paths for EVs by learning value functions based on Dijkstra-generated ground truth [12]. Additional supervised routing models include Q-learning [13], [14], standard Neural Networks (NN) [15], and Deep Graph Neural Networks (DGNN) [10], each trained to generalize routing across variable network sizes and vehicle distributions [6].

The novelty of our work lies in a joint learning framework where infrastructure placement and vehicle routing co-evolve, sharing the learned knowledge. We rigorously test model generalization and adaptation by simulating diverse graphs and loads. Furthermore, our auto-tuned, data-driven ML models require no manual tuning and perform well in new environments. Key contributions include:

- **Integrated Two-Phase Optimization:** A unified framework for joint EV station placement and routing with learning-based models in a modular pipeline.
- **Data-Driven Auto-Tuned ML Models:** DQN, NN, DGNN, and Q-Learning models trained on optimal Dijkstra path data, optimized via supervised learning and reinforcement feedback.
- **ML-Driven Placement Strategies:** Exploration of diverse placement methods (DQN, GNN-spectral analysis, proxy-based scoring) tied to routing outcomes for adaptive planning.
- **Extensive Evaluation:** Detailed simulations on large graphs (100 nodes) and vehicle loads (100) benchmark performance under realistic constraints.
- **Learning and Planning Integration:** Learned routing behaviors evaluate placement policies, creating an efficient planning-control loop.

The remainder of the paper is structured as follows: Section II defines the joint optimization problem for EV charging station placement and vehicle routing and details the system architecture. Section III presents the two-phase methodology, including synthetic data generation, DQN model training, and evaluation metrics. Section IV discusses the simulation results, analyzing the performance of our proposed DQN-based and baseline models across diverse network scenarios. Finally, Section V concludes the paper, highlighting key findings and outlining future research directions, including integration with digital twins and federated learning.

To ensure clarity and consistency across the technical sections, Table I provides a summary of all symbols and notations used throughout this manuscript.

Table I: Comprehensive Notation and Symbol Definitions

Symbol	Definition
$G=(V,E)$	Directed road network graph, where V is the set of nodes and E is the set of directed edges
N	Total number of nodes in the graph ($N=\|V\|$)
K	Number of charging stations to be placed
$S \subset V$	Subset of nodes selected as charging station locations ($\|S\|=K$)
$S^* \subset V$	Optimal set of K charging station locations found by the placement model
$v \in V \setminus S$	A node without a charging station
$s \in S$	A charging station node
$\text{Cost}(v \to s)$	Total energy cost from node v to station s along a shortest path (typically $\sum \text{kWh}(e)$ over path $P_{v\to s}$)
$\mathscr{P}_{v\to s}$	Path from node v to station s
$E(v)$	Energy (kWh) required to reach nearest station in S from node v
$D(v)$	Distance (km) from node v to the nearest station in S
$T(v)$	Travel time (seconds) from node v to the nearest station in S
$H(v)$	Hop count (number of edges) on path from v to the nearest station in S
$\text{kWh}(e)$ or $\text{kWh}(u,u')$	Energy consumption (kWh) of traversing edge $e=(u,u')$
$\text{km}(e)$ or $\text{km}(u,u')$	Distance (km) of edge $e=(u,u')$
$\text{time}(e)$ or $\text{time}(u,u')$	Travel time (seconds) of traversing edge $e=(u,u')$
s^*	Nearest charging station to a given node v from the set S
$V(s)$	Learned value function estimating energy cost-to-go from node s (output of V_θ)
V_θ	Parameterized routing model (e.g., DQN) approximating $V(s)$ using neural network weights θ
θ_P	Parameters (weights) of the DQN placement model Q_P
s_P	Placement state (e.g., binary vector encoding selected station locations, or graph features)
a_P	Placement action (selecting the next node to host a station)
$Q_P(s_P,a_P)$	Q-function for the placement model: expected cumulative reward for action a_P in state s_P
r_P	Reward signal used to evaluate quality of station placement S_K
$\mathscr{S}_P, \mathscr{A}_P$	State and action spaces for placement task
$s_{P,t}$	Placement state at step t (during sequential placement of K stations)
$a_{P,t}$	Placement action at step t
$r_{P,t}$	Reward received after taking placement action $a_{P,t}$ from state $s_{P,t}$
s'_P	Next placement state resulting from action a_P in state s_P
S_{placed}	Set of charging stations placed so far within a single placement episode
s_c	Current node (location of the vehicle) in routing task
s_n	Next-hop node chosen during routing
$\text{Neighbors}(v)$	Set of adjacent (neighboring) nodes to node v
s_{curr}	Current node of a vehicle during path traversal (used in reward calculation for placement and in routing execution)
s_{next}	A potential next-hop node from s_{curr}
$\phi_R(v,S)$	Feature vector extracted for routing state input (e.g., one-hot encoding of v, or learned embedding of v relative to stations S)
$\mathscr{D}_{routing}$	Supervised dataset for routing model training (e.g., pairs of $(\phi_R(v,S), c^*(v,S))$)
$\mathscr{M}_P$	Replay memory for placement model training
ε	Exploration rate in ε-greedy action selection
α	Learning rate (general)
α_P	Learning rate for the DQN placement model
γ	Discount factor (general)
γ_P	Discount factor for the DQN placement model
$\mathscr{L}_{\text{routing}}$	Loss function (e.g., MSE) for training $V_\theta(\phi_R(v,S))$ against target values $c^*(v,S)$
$c^*(v,S)$	Ground truth minimum energy cost from node v to its nearest station in S (from Dijkstra)
$c^*(v,S'_{train})$	Ground truth optimal energy cost from node v to the nearest station in a specific set S'_{train} (used for training routing model)
S'_{train}	Set of fixed station locations used for training the routing model on a specific graph instance G'
$s_n = \arg\min_{s' \in \text{Neighbors}(s_c)} [\text{kWh}(s_c,s') + V_\theta(\phi_R(s',S))]$	Rule for selecting the next hop during routing by minimizing total (immediate + future estimated) cost
AvgEnergy	Average energy consumption across all non-station nodes to reach S
AvgDist	Average distance from non-station nodes to nearest station in S
AvgTime	Average travel time from non-station nodes to station in S
AvgHops	Average hop count over all routes to S
Success Ratio	Ratio of successful routes (within constraints) to total number of requests
$N_{P_episodes}$	Number of training episodes for the DQN placement model
N_{V_epochs}	Number of training epochs for the DQN routing model

II. Problem Description and System Architecture

In modern urban and suburban environments, the rapid growth of EVs creates two intertwined challenges. First, planners must decide *where* to install a fixed number of charging stations so that drivers can reach them conveniently and with minimal energy expenditure. Second, given any chosen set of stations, individual EVs need an efficient way to navigate the road network, selecting routes that consume the least energy while avoiding unnecessary detours or delays. These two tasks, *station placement* and *vehicle routing*, are deeply connected: stations that are poorly sited force vehicles into long, energy-hungry trips, while even an optimal routing policy cannot overcome a suboptimal placement of chargers. Consequently, finding a jointly optimal solution means striking the right balance between broad geographic coverage and energy-efficient routing. This creates a complex combinatorial problem: choosing a handful of locations out

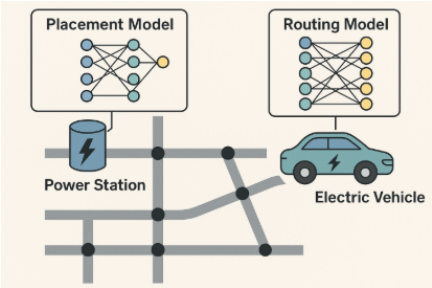

Figure 1: System Model

of many possibilities, and simultaneously guiding EVs along near-optimal paths through a dynamic traffic network. The proposed DQN-based approach tackles these challenges head-on by combining classical graph-theoretic methods with DRL in a two-phase pipeline. First, near-optimal charging-station locations are identified, and subsequently, energy-efficient routing policies are learned. Formally, the coupled two-level optimization is solved as follows:

- *Placement:* choose $S \subset V$, $|S| = K$, to minimize $\frac{1}{|V \setminus S|} \sum_{v \in V \setminus S} \min_{s \in S} \text{Cost}(v \to s)$.
- *Routing:* for fixed S, determine energy-optimal paths from each $v \notin S$ to its nearest $s \in S$, either exactly via Dijkstra or approximately via the learned $V_\theta(\phi_R(v,S))$.

Figure 1 illustrates the complete workflow, from synthetic network generation through placement optimization to trained routing, over a graph-modeled road network, enabling joint station placement planning and vehicle navigation, under a unified framework.

III. Methodology

This section outlines a comprehensive two-phase methodology for optimizing EV charging infrastructure, combining supervised learning and reinforcement learning techniques to improve both station placement and routing efficiency. The pipeline begins with synthetic, strongly connected road network generation and proceeds through feature extraction, model training, and performance evaluation. The learning backbone of the framework leverages Deep Q-Learning (DQL), which extends classical Q-learning by using deep neural networks to approximate the optimal action-value function $Q^*(s,a)$. Instead of relying on tabular Q-values, DQL generalizes over large state-action spaces through a DQN. Stability during training is ensured via experience replay (randomized mini-batch updates) and a target network (periodically updated to stabilize learning). An ε-greedy policy balances exploration and exploitation. This architecture enables DQN to make effective, real-time decisions in high-dimensional EV routing and placement tasks [7], [16], [17]. Overall, the proposed pipeline supports a scalable and generalizable approach to EV infrastructure planning, integrating simulation-based datasets, ML model training, and robust evaluation for both placement and routing tasks. Note that the placement DQN (stage 1) will be trained using a routing-optimized pre-trained network (done in stage 2).

A. Dataset Generation

a) Road Network Synthesis

Synthetic urban road networks are modeled as directed graphs $G=(V,E)$, where nodes V represent intersections, and

edges E represent directed road segments. Each edge $e \in E$ is annotated with distance ($\text{km}(e)$) and energy consumption ($\text{kWh}(e) = 0.2 \times \text{km}(e)$). To ensure strong connectivity, graphs are generated from a base ring topology with additional random and targeted edges. Diverse topologies are constructed using varying node counts and random seeds to simulate different urban layouts.

b) Placement Feature Dataset

For each generated graph, a greedy Dijkstra-based heuristic is used to select an initial placement of K charging stations. The first station is placed at the node with the minimum total distance to all others. Subsequent stations are added iteratively to minimize the overall nearest-station distance across all nodes. This process ensures spatial diversity and reduced average travel cost. For each placement configuration S, a feature vector is extracted, capturing the impact of station locations on network accessibility. Specifically, for every non-station node $v \in V \setminus S$, the shortest path to the nearest station $s^* \in S$ is computed using Dijkstra's algorithm, from which several statistics are derived. Energy features include the average and standard deviation of energy required to reach the nearest station: $\text{AvgEnergy} = \frac{1}{|V \setminus S|} \sum_{v \in V \setminus S} E(v)$. Distance metrics include the average and standard deviation of the shortest path lengths: $\text{AvgDist} = \frac{1}{|V \setminus S|} \sum_{v \in V \setminus S} D(v)$. Hop count statistics (AvgHops) and time-based metrics (AvgTime) are also computed, with time estimated from distance and average speed. Together, these aggregated statistics describe the overall accessibility and energy efficiency of the placement. Additional graph-theoretic features, such as coverage within 10km and centrality metrics (e.g., degree, betweenness/proximity), are included to enrich spatial analysis. The resulting vectors are labeled with multi-label binary targets indicating selected station nodes, forming a supervised training dataset.

c) Routing (Path-Finding) Dataset

To train routing models, a comprehensive dataset is generated using synthetically created graphs of various sizes and topologies. Each graph includes edge-level annotations for distance ($\text{km}(e)$), energy cost ($\text{kWh}(e)$), and estimated travel time ($\text{time}(e)$), assuming average vehicle speed. Charging station placements S are again derived using a greedy Dijkstra-based heuristic. For each non-station node v, the nearest station s^* is identified by minimizing the total energy cost:

$$s^* = \arg\min_{s \in S} \text{Cost}(v \to s),$$

where $\text{Cost}(v \to s) = \sum_{(u,u') \in \mathscr{P}_{v \to s}} \text{kWh}(u,u')$, with $\mathscr{P}_{v \to s}$ the shortest-path returned by Dijkstra's algorithm. The optimal path $\mathscr{P}_{v \to s^*}$ is extracted, and the following metrics are computed:

$$E(v) = \sum_{(u,u') \in \mathscr{P}_{v \to s^*}} \text{kWh}(u,u'), \tag{1}$$

$$D(v) = \sum_{(u,u') \in \mathscr{P}_{v \to s^*}} \text{km}(u,u'), \tag{2}$$

$$T(v) = \sum_{(u,u') \in \mathscr{P}_{v \to s^*}} \text{time}(u,u'), \tag{3}$$

$$H(v) = |\mathscr{P}_{v \to s^*}|. \tag{4}$$

where $|\mathscr{P}_{v \to s^*}|$ denotes the number of edges (hops) in the path from v to its nearest charging station s^*. Each sample is stored as a tuple $[v, s^*, E(v), D(v), T(v), H(v)]$, used to train supervised models for routing cost approximation. This dataset supports the learning of DQN V_θ capable of generalizing routing behavior across topologies, eliminating the need for explicit shortest path computation at inference time.

B. Placement and Routing Model Training and Architecture

This section presents the detailed training methodologies and architectural considerations of the proposed Deep Q-Network (DQN) framework designed for the joint optimization of EV charging station placement and vehicle routing. Initially, we describe the formulation and training procedures for the placement model, followed by the routing model. For each model, we thoroughly outline state and action representations, reward mechanisms, neural network architectures, and the supervised and reinforcement learning approaches employed for effective training and evaluation. Collectively, these components facilitate scalable, robust, and data-driven optimization strategies for EV charging infrastructure and efficient route planning within intricate network environments.

1) Placement Model

a) DQN Placement Model Training

The DQN placement model is trained using reinforcement learning to select the optimal set of K charging station locations within a given graph. The environment state is encoded as a binary vector $s_P \in \{0,1\}^N$, where each element indicates whether a node currently hosts a station. At each timestep, the agent selects an action $a_P \in \{1, \ldots, N\}$ corresponding to the next node where a station should be placed. The placement process proceeds sequentially until K stations are selected, forming a complete placement configuration. Dijkstra's algorithm is used to generate the "expert" placements for the supervised pre-training phase: for each sampled graph, a greedy K-station configuration is computed by iteratively selecting the node that minimizes the total shortest-path distance from that node to all other nodes. Overall, the DQN placement model was trained using 70% of the generated graph instances, while the remaining 30% were reserved for evaluation on unseen topologies to assess generalization and placement effectiveness.

b) DQN Placement Model

The model is implemented as a multi-layer perceptron with input size N, hidden layers with ReLU activations, and an output layer of size N representing Q-values for all node actions. The training employs experience replay and ε-greedy exploration to balance exploitation and exploration. Once convergence is achieved, the agent is able to recommend near-optimal station placements for unseen graphs without needing to simulate all possible configurations. The strategic placement of EV charging stations is modeled as a sequential decision-making process, addressed by a DQN-based reinforcement learning agent. This agent learns a policy to select an optimal set of K charging station locations from the available nodes V in the graph $G = (V, E)$. The core components of this RL approach are defined as follows:

State Representation: The state $s_P \in \mathscr{S}_P$ for the placement agent encapsulates information about the graph's topology and the current partial placement of charging stations. This can be represented, for instance, by a binary vector of length $|V|$ indicating which nodes have already been selected to host stations, potentially augmented with a counter for the number

of stations placed so far, or by more complex graph embedding features that capture the overall network configuration.

Action Space: At each decision step t in the placement process (where t ranges from 1 to K, the total number of stations to be placed), the agent selects an action $a_P \in \mathscr{A}_P$. The action space $\mathscr{A}_P$ consists of all graph nodes V that have not yet been chosen in the current partial placement S_{t-1} (i.e., $\mathscr{A}_P = V \setminus S_{t-1}$). Executing an action a_P corresponds to selecting the chosen node to host the next charging station.

Reward Function: The reward r_P is critical for guiding the placement agent's learning process. Typically, a significant reward (or penalty) is provided only upon the completion of a full placement, i.e., after all K stations have been sited. This terminal reward reflects the overall quality of the chosen set of K station locations, $S_K = \{a_{P,1}, \ldots, a_{P,K}\}$. The quality is quantified by the efficiency of routing vehicles to these stations. Specifically, the reward can be defined as the negative of the average routing energy (or a similar metric like travel time or unreachability penalty) calculated for a representative set of vehicles or all non-station nodes when routed to their nearest station in S_K. These routing costs can be estimated using a pre-trained DQN routing model V_θ, albeit computationally intensive, with ground-truth evaluation during RL training. The reward r_P is critical for guiding the placement agent's learning process. So, the reward is formulated as

$$r_P = -\frac{1}{|V \setminus S_K|} \sum_{v \in V \setminus S_K} \text{TrueRoutingCost}(v, S_K),$$

where

$$\text{TrueRoutingCost}(v, S_K) = \min_{s \in S_K} \min_{\mathscr{P}: v \to s} \sum_{(u,u') \in \mathscr{P}} \text{kWh}(u, u'),$$

which represents the minimum cumulative energy along any path $\mathscr{P}$ from node v to one of the placed stations in S_K, computed ideally (e.g., via Dijkstra). A deep neural network (with parameters θ_P) is utilized to approximate the optimal action-value function $Q_P^*(s_P, a_P)$, representing the maximum expected cumulative reward for taking action a_P in state s_P and following the optimal policy thereafter. The network maps states s_P to Q-values for all possible actions. It is trained by minimizing the temporal difference error, with updates guided by the Bellman equation:

$$Q_P(s_{P,t}, a_{P,t}) \leftarrow Q_P(s_{P,t}, a_{P,t}) \tag{5}$$

$$+ \alpha_P \Big[r_{P,t} + \gamma_P \max_{a'} Q_P(s'_P, a') - Q_P(s_{P,t}, a_{P,t}) \Big] \tag{6}$$

where α_P is the learning rate and γ_P is the discount factor for placement. Standard RL techniques such as experience replay (storing and sampling transitions (s_P, a_P, r_P, s'_P)) and an ε-greedy exploration strategy are employed during training, which proceeds over numerous episodes, each involving the placement of K stations.

2) Routing Model

While our placement model relies on Q-learning with discrete actions $Q_P(s_P, a_P)$, the routing model instead estimates a scalar value function $V_\theta(\phi_R(s, S))$, as the objective is to evaluate expected energy cost from a node to its nearest station rather than choosing between multiple actions from that node.

a) DQN Routing Model Training

The routing model is trained as a value-based DRL model that approximates the state-value function $V(s)$, where each state s (represented by features $\phi_R(s, S)$) represents a node in the graph and the value $V_\theta(\phi_R(s, S))$ denotes the model's estimate of the minimum cumulative energy required to reach the nearest charging station in S. Specifically, for each node v, the target value $c^*(v, S)$ (the true minimum energy cost to the nearest station in a given fixed set of stations S) is:

$$c^*(v, S) = \min_{s \in S} \min_{\mathscr{P}: v \to s} \sum_{(u,u') \in \mathscr{P}} \text{kWh}(u, u'),$$

i.e., the true minimum cumulative energy along an optimal path $\mathscr{P}$ from node v to its nearest station in a given set S. The input to the model V_θ is a feature vector $\phi_R(v, S)$. The output is a scalar $V_\theta(\phi_R(v, S))$ estimating $c^*(v, S)$. Training data is generated using Dijkstra's algorithm to compute optimal energy paths from each non-station node to its nearest station. Let $c^*(v, S)$ be the target cost from node v. Overall, the DQN routing model V_θ was trained on 70% of the synthetic graphs with known optimal routing costs, while the remaining 30% were held out for evaluation to validate the model's ability to generalize and accurately predict routing behavior on unseen network topologies.

b) DQN Routing Model

The DQN model V_θ is implemented as a multi-layer perceptron (MLP) with two hidden layers. The first layer has $n_1 \in [32, 192]$ units and the second layer has $n_2 \in [16, 96]$ units, both followed by ReLU activations and dropout regularization. The output layer is a linear projection producing a scalar value. The learning rate is optimized in the range $\in [10^{-5}, 10^{-2}]$. Hyperparameters—including hidden layer sizes and learning rate—are optimized using Bayesian optimization. Training employs early stopping and adaptive learning rate scheduling to ensure convergence. Once trained, the DQN model V_θ is reused across different graph scenarios to evaluate placement strategies and execute real-time routing decisions. The routing model utilizes a DQN structure (V_θ) to approximate the state-value function for all nodes (states) $s \in V$ in the graph. This function, $V_\theta(\phi_R(s, S))$, represents the estimated minimum future energy cost to reach a charging station in S starting from node s.

State Representation: The state is represented by features $\phi_R(s_c, S)$ for the current node $s_c \in V$ relative to the set of stations S. The model input representing the state allows the network to predict $V_\theta(\phi_R(s_c, S))$. The model is trained to predict this value for every node. For instance, the model V_θ takes $\phi_R(s_j, S)$ as input and outputs its prediction of the cost-to-go from s_j. The model is trained in a supervised manner. Ground truth energy costs ($c^*(v, S)$) from each node v to the nearest charging station in S are obtained via Dijkstra's algorithm. The training objective is to minimize the Mean Squared Error (MSE) between the model's predicted cost-to-go $V_\theta(\phi_R(v, S))$ and the target cost $c^*(v, S)$:

$$\mathscr{L}_{\text{routing}} = \frac{1}{|V|} \sum_{v \in V} (V_\theta(\phi_R(v, S)) - c^*(v, S))^2 \tag{7}$$

where $c^*(v, S)$ is the true minimum energy cost from node v to its nearest charging station in S, computed via Dijkstra's algorithm.

Action Space (for path selection): During routing, when a

vehicle is at a current node s_c, the available actions consist of moving to any adjacent node $s' \in \text{Neighbors}(s_c)$. The selection of the optimal action (i.e., the next hop s_n) is guided by the learned value function V_θ.

Reward Function (implicit in supervised training): For the training of the V_θ function itself, there isn't an explicit reward signal in the traditional reinforcement learning sense of $R(s,a,s')$. Instead, the "reward" is implicitly captured by the supervised learning target $c^*(v,S)$ (the optimal cost-to-go). The model is "rewarded" for predictions $V_\theta(\phi_R(v,S))$ that are close to $c^*(v,S)$ by having a lower MSE loss. During path selection, the "reward" for taking an action is the negative of the immediate $\text{kWh}(s_c,s')$. For decision-making during routing (i.e., path selection), when the vehicle is at the current node s_c, the model evaluates the cost of moving to each neighbor s'. The next hop, s_n, is chosen using the rule that minimizes the sum of the immediate transition cost and the estimated future cost from the next state:

$$s_n = \arg\min_{s' \in \text{Neighbors}(s_c)} \left[\text{kWh}(s_c,s') + V_\theta(\phi_R(s',S))\right] \quad (8)$$

Here, $\text{kWh}(s_c,s')$ is the actual energy cost of traversing the edge from s_c to s', and $V_\theta(\phi_R(s',S))$ is the model's predicted cost-to-go from the neighboring state s' relative to stations S. This approach enables the agent to select paths that minimize the cumulative energy consumption by combining immediate transition costs with long-term estimates of future costs, effectively mimicking classical shortest-path algorithms while enabling generalization to unseen graph topologies.

C. The DQN Approach Algorithm

In Algorithm 1, we show the proposed DQN approach. The overall process involves first training the DQN Routing Model (V_θ) as detailed in Phase 2. This trained V_θ is then utilized as a component within Phase 1 for training the DQN Placement Model (Q_P).

Algorithm 1 Two-Phase DQN Framework for EV Charging Station Placement and Routing

Require: Graph characteristics (for $G=(V,E)$), station count K, training parameters for DQN Placement Model Q_P (e.g., episodes $N_{P_episodes}$, α_P, γ_P), training parameters for DQN Routing Model V_θ (e.g., dataset size, epochs N_{V_epochs})
Ensure: Optimal Placement S^*, Trained DQN Placement Model Q_P, Trained DQN Routing Model V_θ

// Phase 1: Train DQN Placement Model (Q_P) to learn optimal placement actions
▷ This phase utilizes the pre-trained DQN Routing Model V_θ (from Phase 2) for reward calculation.
Initialize DQN Placement Model $Q_P(s_P,a_P)$ with parameters θ_P.
Initialize experience replay memory $\mathscr{M}_P$.
for episode = 1 to $N_{P_episodes}$ **do**
  Generate/select a graph $G=(V,E)$ for placement task.
  $S_{placed} \leftarrow \emptyset$. Current placement state $s_P \leftarrow \text{GetInitialPlacementState}(G, S_{placed})$.
  for step = 1 to K **do** ▷ Sequentially place K stations
    Select action a_P (node to place next station) from s_P using ε-greedy policy on Q_P.
    $S_{placed} \leftarrow S_{placed} \cup \{a_P\}$.
    $s'_P \leftarrow \text{GetNextPlacementState}(G, S_{placed})$.
    if step == K **then** ▷ Full placement S_{placed} achieved by agent
      /* Calculate reward r_P using the pre-trained DQN Routing Model V_θ */
      Let $L_{costs_S_{placed}} \leftarrow []$.
      for all non-station nodes $v \in V \setminus S_{placed}$ **do**
        Simulate path $\mathscr{P}_{v \to S_{placed}}$ from v to nearest station in S_{placed} using routing policy derived from V_θ: next hop $s'' = \arg\min_{s_{next} \in \text{Neighbors}(s_{curr})} (\text{kWh}(s_{curr}, s_{next}) + V_\theta(\phi_R(s_{next}, S_{placed})))$.
        Append cost $E(\mathscr{P}_{v \to S_{placed}})$ to $L_{costs_S_{placed}}$.
      end for
      $r_P = -\text{Mean}(L_{costs_S_{placed}})$ if $L_{costs_S_{placed}}$ else LargeNegativePenalty.
      Store transition $(s_P, a_P, r_P, s'_P, \text{done=true})$ in $\mathscr{M}_P$.
    else
      $r_P = 0$ (or small intermediate reward).
      Store transition $(s_P, a_P, r_P, s'_P, \text{done=false})$ in $\mathscr{M}_P$.
    end if
    $s_P \leftarrow s'_P$.
    Sample minibatch from $\mathscr{M}_P$; update Q_P using Bellman eq: $Q_P(s_P,a_P) \leftarrow Q_P(s_P,a_P) + \alpha_P[r_P + \gamma_P \max_{a'} Q_P(s'_P,a') - Q_P(s_P,a_P)]$.
    if step == K **then break**;
    end if
  end for
end for
$S^* \leftarrow$ Determine optimal placement by following learned policy of Q_P on target graph(s).

// Phase 2: Train DQN Routing Model (V_θ) to approximate cost-to-go
Initialize an empty dataset $\mathscr{D}_{routing}$ for V_θ.
Generate a set of training instances: graphs $G' = (V', E')$.
for all each training graph $G' = (V', E')$ **do**
  Determine fixed station placements S'_{train} for G' ▷ e.g., using a greedy Dijkstra-based heuristic as per Sec. III.A.
  for all non-station nodes $v \in V' \setminus S'_{train}$ **do**
    Compute optimal energy cost $c^*(v, S'_{train})$ from v to nearest station in S'_{train} (Dijkstra).
    Extract state features $\phi_R(v, S'_{train})$ for routing model input.
    Add $(\phi_R(v, S'_{train}), c^*(v, S'_{train}))$ to $\mathscr{D}_{routing}$.
  end for
end for
Train DQN Routing Model $V_\theta(\phi_R(s,S))$ (approximating state-value function) using $\mathscr{D}_{routing}$ by minimizing MSE loss ($\mathscr{L}_{\text{routing}}$).
return S^*, Q_P, V_θ.

IV. Simulation Results and Analysis

This section presents the evaluation of the proposed DQN framework for EV charging station placement and vehicle routing. Simulations were conducted using classical and learning-based models. Key performance metrics included energy consumption, travel time, hop count, and success ratio. Our proposed DQN-based approach consistently outperformed Q-Learning, DGNN, and NN baselines in both placement and routing phases, achieving the lowest energy usage and highest route reliability. The simulation setup ensured fair comparison, and generalization tests confirmed DQN's robustness on unseen graph scenarios. Numerical precision in results is two decimal places for clarity.

A. Simulation Setup

The simulation environment was designed to evaluate classical and learning-based strategies for EV charging station placement and routing under controlled, reproducible conditions. Each experiment was conducted on graphs consisting of 100 nodes, with edges generated probabilistically using a 0.30 edge creation threshold and distances ranging from 1 to 20 kilometers. The simulation included 100 vehicles and allocated 3 charging stations per run. Routing algorithms tested included Dijkstra, our proposed DQN, Q-Learning, and DGNNs. For learning-based models, the DRL agents were trained for a minimum of 50 episodes with early stopping enabled after 10 stagnant episodes. Supervised models were trained using a dataset of 1000 synthetic graph samples. Key evaluation metrics collected throughout the simulations included energy consumption, travel time, hop count, routing distance, and success rate. These settings ensured a fair and scalable comparison across all evaluated approaches.

B. Results of the First Step: Charging Station Placement

Table II (results to two decimal places) summarizes four charging station placement strategies (100-node network, 3 stations), considering average energy, travel time, and nearest-station distance. DQN placement demonstrated superior energy efficiency (1.24 kWh/EV), with the NN method close (1.27 kWh). In contrast, DGNN and QL consumed significantly more (1.58 kWh and 1.78 kWh, respectively), underscoring DQN and NN's advantage in energy-reducing configurations. However, a time-energy trade-off emerged: DQN's average travel time (10.21s) was slightly above NN (10.00s) and considerably higher than DGNN (6.01s) and QL (6.67s). This suggests DGNN and QL prioritize speed over energy, while DQN and NN favor efficiency with modestly longer paths. For spatial coverage, DQN and NN achieved identical average distances to the nearest station (6.34 km), outperforming DGNN (7.09 km) and QL (8.89 km), indicating more centralized and accessible placements. Overall, DQN offers a balanced,

robust solution for energy-efficient and spatially equitable charging infrastructure. While DGNN and QL provide faster routes, their higher energy costs and poorer accessibility make them less suitable for energy-sensitive applications. NN's strong performance further validates learning-based placement strategies.

Table II: Step 1: Placement Results of Charging Stations

Placement Strategy	Average Energy (kWh)	Average Time (s)	Average Distance (km)
DQN	1.24	10.21	6.34
NN	1.27	10.00	6.34
DGNN	1.58	6.01	7.09
QL	1.78	6.67	8.89

C. Results of the Second Step: Routing Toward Charging Stations

Table III evaluates routing strategies (100 vehicles, fixed stations; results to two decimal places). DQN routing consistently outperformed others, achieving the lowest energy (1.26 kWh), highest success (96.67%, 2.9/3 routes), low hops (1.98), and moderate time (9.95s), demonstrating efficient and reliable path learning. NN performed similarly (1.28 kWh, 93.33% success) with minor trade-offs. DGNN was faster but less energy-efficient (1.39 kWh, 90% success). QL was least effective (1.43 kWh, 86.67% success), prioritizing speed over other metrics. DQN offers the best overall balance, with NN and DGNN as alternatives and QL falling short.

Table III: Step 2: Routing Towards Charging Stations

Approach	Average Energy (kWh)	Average Time (s)	Average Distance (km)	Average Hops	Average Successful Routes	Average Success Ratio
DQN	1.26	9.95	6.32	1.98	2.9	0.97
NN	1.28	10.05	7.02	2.19	2.8	0.93
DGNN	1.39	6.28	7.16	2.03	2.7	0.90
QL	1.43	6.72	7.36	2.24	2.6	0.87

Figure 2 illustrates the average energy consumption (in kWh) per training episode across five routing models: the proposed DQN, Q-Learning, DGNN, NN, and Random baseline. The DQN consistently outperforms all other methods, rapidly converging to the lowest energy levels below 70 kWh. Q-Learning and DGNN show slower convergence and higher steady-state energy consumption, plateauing around 80 kWh and 75 kWh respectively. The NN model stabilizes above 85 kWh, while the Random baseline remains the highest with no learning progression, exceeding 120 kWh throughout. The results confirm DQN's superior energy efficiency and learning stability over time, highlighting its advantage in optimizing energy-aware EV routing policies.

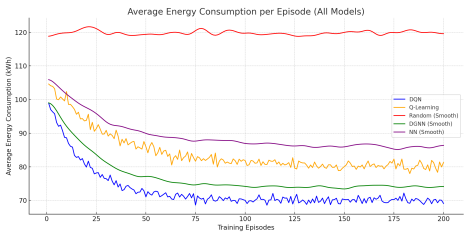

Figure 2: Average Energy Consumption (kWh) per Episode for All Routing Models

V. Conclusions and Future Work

This paper proposed a two-stage DQN framework for optimizing EV charging station placement and routing. The placement model selected energy-efficient station locations, while the routing model minimized per-vehicle travel cost using learned cost-to-go estimates. Evaluations showed that DQN consistently outperformed Q-Learning, Random Walk, neural networks, and DGNNs, achieving the lowest energy consumption and highest route success rates. The placement strategy approximated optimal deployments via routing-based evaluation, and the routing model enabled scalable, near-optimal decisions with Dijkstra-like accuracy. Future work will incorporate dynamic traffic and demand modeling, explore federated learning for privacy-preserving collaboration, integrate stochastic battery and charging models, and validate the approach in digital twin platforms and real-world testbeds.

Acknowledgment

This work has received funding from the European Union's Horizon 2020 Research and Innovation Programme under Grant Agreement No. 739578, the METACITIES project under Grant Agreement No. 101087257, and the Government of the Republic of Cyprus through the Deputy Ministry of Research, Innovation and Digital Policy.

References

[1] International Energy Agency, "Global EV Outlook 2023," 2023. [Online]. Available: https://www.iea.org/reports/global-ev-outlook-2023

[2] X. Li and et al., "Charging infrastructure planning for electric vehicles: A review," *Renewable and Sustainable Energy Reviews*, vol. 120, p. 109611, 2020.

[3] S. Guo and et al., "Urban electric vehicle charging station placement considering traffic flow and power grid constraints," *Applied Energy*, vol. 253, p. 113554, 2019.

[4] J. Kim and et al., "Ev charging station placement and routing: A comprehensive review," *Journal of Cleaner Production*, vol. 287, p. 125023, 2021.

[5] Y. Chen and et al., "Routing electric vehicles with energy consumption and time windows," *Transportation Research Part E: Logistics and Transportation Review*, vol. 118, pp. 51–69, 2018.

[6] J. Smith and A. Jones, "Dynamic network environments for electric vehicle routing," *Journal of Smart Mobility*, vol. 5, no. 1, pp. 45–60, 2022.

[7] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[8] Y. Tang, F. Yang, and Y. Chen, "Reinforcement learning based optimal routing and charging strategy for electric vehicles in urban networks," *Transportation Research Part C: Emerging Technologies*, vol. 125, p. 103070, 2021.

[9] W. Zheng, S. Guo, and Q. Wang, "Integrated evolutionary and deep learning model for efficient electric vehicle charging infrastructure planning," *Energy*, vol. 245, p. 123237, 2022.

[10] H. Li, D. Zhang, and X. Xu, "Charging station availability prediction using graph neural networks for electric vehicles," *IEEE Transactions on Intelligent Transportation Systems*, vol. 22, no. 5, pp. 2765–2777, 2021.

[11] Y. Wu, K. Li, and J. Wang, "Optimal planning of electric vehicle charging stations considering coverage and grid impact," *IEEE Transactions on Smart Grid*, vol. 11, no. 4, pp. 3121–3131, 2020.

[12] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.

[13] C. J. C. H. Watkins, "Learning from delayed rewards," Ph.D. dissertation, University of Cambridge, 1989.

[14] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[15] J. Sun, R. Huang, and M. Li, "Deep learning based electric vehicle charging demand forecasting using lstm with multisource data fusion," *Energy Reports*, vol. 9, pp. 877–886, 2023.

[16] F. Ye, J. Ma, G. Zheng, Y. Zhang, and X. Du, "Deep reinforcement learning for vehicle routing problem with time windows," *IEEE Access*, vol. 8, pp. 143 824–143 835, 2020.

[17] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Bharath, "A brief survey of deep reinforcement learning," *IEEE Signal Processing Magazine*, vol. 34, no. 6, pp. 26–38, 2017.